

Ejercicios REST — Nivel Medio

Prerequisito: familiaridad con NestJS, inyección de dependencias, y el patrón Repository/Gateway que usa la API. La API base tiene `products` (CRUD in-memory) y `users` (proxy a JSONPlaceholder).

Ejercicio 1 — Módulo Categories con relación a Products

Objetivo: Crear un módulo `categories` y asociar cada producto a una categoría.

Entidad Category:

```
{ id: number, name: string }
```

Cambios en Product:

```
{ id: number, name: string, price: number, categoryId: number }
```

Endpoints requeridos:

```
GET    /categories
GET    /categories/:id
POST   /categories
DELETE /categories/:id
```

```
GET    /categories/:id/products  ← productos de esa categoría
```

Reglas: - No se puede eliminar una categoría que tenga productos asociados → 409 Conflict - Al crear un producto, si `categoryId` no existe → 400 Bad Request - Seguir el mismo patrón de módulo que `ProductsModule`: controller → service → repository

Archivos a crear: - `src/categories/` con su propio módulo, controller, service, repository e interface

Ejercicio 2 — Paginación en listados

Objetivo: Implementar paginación basada en cursor (offset/limit) para `GET /products` y `GET /categories`.

Query params:

```
GET /products?page=2&limit=5
```

Respuesta esperada:

```
{
  "data": [...],
}
```

```

    "meta": {
      "page": 2,
      "limit": 5,
      "total": 23,
      "totalPages": 5
    }
  }
}

```

Consideraciones: - limit máximo: 50. Si se pasa uno mayor, usar 50. - page mínimo: 1. Si se pasa 0 o negativo, usar 1. - Crear un tipo genérico reutilizable `PaginatedResult<T>` en un archivo compartido

Ejercicio 3 — Reemplazar el repositorio in-memory por SQLite con TypeORM

Objetivo: Migrar `InMemoryProductsRepository` a una base de datos real sin cambiar la interfaz del repositorio.

Pasos: 1. Instalar `@nestjs/typeorm`, `typeorm`, `sqlite3` 2. Configurar `TypeOrmModule` en `AppModule` con `SQLite` 3. Crear una entidad `ProductEntity` con decoradores `TypeORM` 4. Crear `TypeOrmProductsRepository` que implemente la misma interfaz `ProductsRepository` 5. Reemplazar el `useClass` en `ProductsModule` sin tocar el `service` ni el `controller`

La interfaz del repositorio NO debe cambiar. Ese es el punto del ejercicio.

Verificación: El `controller` y el `service` no deben tener ninguna referencia a `TypeORM`.

Ejercicio 4 — Validación con class-validator

Objetivo: Agregar validación estricta a todos los endpoints que reciben `body`.

Instalar: `class-validator`, `class-transformer`

Activar: `ValidationPipe` global en `main.ts`

Reglas para `CreateProductInput`: - `name`: string, mínimo 2 caracteres, máximo 100, requerido - `price`: número positivo, mayor a 0, requerido - `stock`: entero, mínimo 0, requerido

Reglas para `UpdateProductInput`: - Todos los campos opcionales, pero si se envían deben cumplir las mismas restricciones

Comportamiento esperado:

`POST /products` `body`: { `name`: "A", `price`: -10 }

→ 400 Bad Request con detalle de cada error de validación

Transformar los inputs en clases (`class-transformer`) en lugar de interfaces para que los decoradores funcionen.

Ejercicio 5 — Gateway alternativo para usuarios

Objetivo: Crear un segundo gateway de usuarios que lea desde un archivo JSON local, y poder elegir cuál usar mediante una variable de entorno.

Contexto: La API tiene el patrón Gateway para abstraer fuentes de datos externas. Este ejercicio explota esa arquitectura.

Pasos: 1. Crear `src/users/gateways/local-users.gateway.ts` que lee desde `src/users/data/users.json` 2. Poblar ese JSON con al menos 5 usuarios de prueba 3. En `UsersModule`, usar un factory provider que elija el gateway según `process.env.USERS_SOURCE`: - "local" → `LocalUsersGateway` - cualquier otro valor → `JsonPlaceholderUsersGateway`

El controller y el service no deben cambiar.

```
// Ejemplo del factory provider
{
  provide: USERS_GATEWAY,
  useFactory: () => {
    return process.env.USERS_SOURCE === 'local'
      ? new LocalUsersGateway()
      : new JsonPlaceholderUsersGateway();
  }
}
```